

Notification Service API - Smoke Testing Guide

Purpose

Verify the application from startup through product, cart, order, notification, repository persistence, and Gmail SMTP email sending.
Run these steps in order after pulling the latest code.

Current architecture checkpoint

- Controllers are thin and only shape HTTP responses.
- Services coordinate workflows and map DTOs.
- Domain entities own business rules and state changes.
- Persistence/Interfaces contains repository contracts.
- Persistence/Concrete contains EF Core repository implementations.
- Repositories use NotificationDbContext for queries, add/remove operations, and SaveChanges.
- LoggingEmailSender logs first, then delegates to GmailSmtpEmailSender for real email delivery.

Prerequisites

1. SQL Server is available for the NotificationServiceDb connection string in appsettings.json.
2. Gmail SMTP settings are configured before testing real email delivery.
Required SmtpEmail values: Username, Password, FromAddress. Use a Gmail App Password.
3. The test recipient addresses in appsettings.json are valid for your environment.

Step 1 - Start the app

Command: dotnet run

Expected: The app starts without errors, EF Core applies pending migrations, DI resolves services and repositories, and Swagger is available.

Open: <https://localhost:<port>/swagger> or <http://localhost:<port>/swagger> using the port shown in the console.

Step 2 - Verify Swagger loads

Request: GET /swagger

Expected: Swagger UI opens and lists Products, Cart, Orders, and Notifications endpoints.

Step 3 - Verify product catalog

Request: GET /api/products

Expected: 200 OK with product records containing id, name, shortDescription, description, price, quantityAvailable, and imageUrl.

Repository path: ProductsController -> ProductService -> IProductRepository -> ProductRepository -> NotificationDbContext.

Step 4 - Verify product detail

Request: GET /api/products/1

Expected: 200 OK with one product. If id does not exist, expect 404 Not Found.

Step 5 - Create a product

Request: POST /api/products

Body example:

```
{ "name": "Smoke Test Product", "shortDescription": "Created during smoke test", "description":
```

"Temporary product for end-to-end validation", "price": 99.00, "quantityAvailable": 10,
"imageUrl": "https://example.com/product.jpg" }
Expected: 201 Created with the created product and a generated id.
Domain behavior: Product.Create validates required text, non-negative price, and non-negative quantityAvailable.

Step 6 - Update the product

Request: PUT /api/products/{createdProductId}

Body example:

```
{ "name": "Smoke Test Product Updated", "shortDescription": "Updated during smoke test",  
  "description": "Updated product for end-to-end validation", "price": 109.00,  
  "quantityAvailable": 8, "imageUrl": "https://example.com/product-updated.jpg" }
```

Expected: 200 OK with updated values. Unknown product id returns 404 Not Found.

Domain behavior: Product.Update owns product state changes and validation.

Step 7 - Add product to cart

Request: POST /api/cart/items

Body example: { "visitorEmail": "bmacha2015@gmail.com", "productId": 1, "quantity": 2 }

Expected: 200 OK with cart items, line-item subtotal, and totalAmount.

Validation check: quantity <= 0 returns 400 Bad Request. Unknown productId returns 404 Not Found.

Repository path: CartController -> CartService -> ICartRepository -> CartRepository -> NotificationDbContext.

Domain behavior: CartItem.Create or CartItem.AddQuantity enforces positive quantities.

Step 8 - View the cart

Request: GET /api/cart/bmacha2015@gmail.com

Expected: 200 OK with the visitor email, cart items ordered by product name, subtotals, and totalAmount.

Step 9 - Update a cart item quantity

Request: PUT /api/cart/items/{cartItemId}

Body example: { "quantity": 3 }

Expected: 200 OK with updated cart totals. Unknown cart item id returns 404 Not Found.

Domain behavior: CartItem.UpdateQuantity enforces positive quantities and updates UpdatedAtUtc.

Step 10 - Place an order from the cart

Request: POST /api/orders

Body example: { "visitorEmail": "bmacha2015@gmail.com" }

Expected: 201 Created. The cart is converted into an order, order items are persisted, and the cart is cleared.

Repository path: OrdersController -> OrderService -> IOrderRepository -> OrderRepository -> NotificationDbContext.

Domain behavior: Order.CreateFromCart builds the order, OrderItem.Create snapshots product name and price, and totals are calculated.

Notification behavior: NotificationDispatcher creates admin and visitor notifications through INotificationRepository.

Email behavior: LoggingEmailSender logs, then GmailSmtpEmailSender sends through

smtp.gmail.com:587 when SmtEmail is configured.

Step 11 - Verify order history and order detail

Request: GET /api/orders?visitorEmail=bmacha2015@gmail.com

Expected: 200 OK with the new order in descending creation order.

Request: GET /api/orders/{orderId}

Expected: 200 OK with order statuses, totalAmount, and order item details.

Step 12 - Update order status

Request: PATCH /api/orders/{orderId}/status

Body example:

```
{ "orderStatus": "Preparing", "paymentStatus": "Paid", "fulfillmentStatus": "Packing",  
  "deliveryStatus": "WaitingForRider" }
```

Expected: 200 OK with updated statuses and updatedAtUtc changed.

Domain behavior: Order.UpdateStatuses owns the status state changes.

Notification behavior: admin and visitor status-update notifications are created and emailed.

Step 13 - Verify notifications

Request: GET /api/notifications?recipientEmail=bmacha2015@gmail.com

Expected: 200 OK with visitor notifications ordered by newest first.

Request: GET /api/notifications?orderId={orderId}

Expected: 200 OK with notifications linked to the order.

Repository path: NotificationsController -> NotificationService -> INotificationRepository -> NotificationRepository -> NotificationDbContext.

Step 14 - Verify notification CRUD

Request: POST /api/notifications with recipientEmail, channel, subject, body, and optional orderId.

Expected: 201 Created. Required recipientEmail, subject, and body are enforced by the Notification domain entity.

Request: PUT /api/notifications/{notificationId}

Expected: 200 OK with updated notification fields and read state.

Request: DELETE /api/notifications/{notificationId}

Expected: 204 No Content. Unknown notification id returns 404 Not Found.

Step 15 - Verify email delivery

Check application logs for queued and sent email messages from LoggingEmailSender.

Check the Gmail inboxes for the admin and visitor recipients if real SMTP credentials were configured.

If SmtEmail settings are incomplete, email sending should fail fast with an SMTP configuration error.

Step 16 - Final database checks

Verify SQL Server tables: Products, CartItems, Orders, OrderItems, Notifications, and __EFMigrationsHistory.

Expected: created/updated products exist, cart is cleared after order placement, order items remain, notifications are linked to the order.

Step 17 - Repository wiring check

Confirm Program.cs registers:

- ICartRepository -> CartRepository
- IOrderRepository -> OrderRepository
- IProductRepository -> ProductRepository
- INotificationRepository -> NotificationRepository

Expected: app startup succeeds and API workflows resolve repository dependencies through DI.

Pass criteria

The app starts, Swagger loads, repositories resolve, the product/cart/order workflow succeeds, notifications are persisted, and configured Gmail SMTP email is sent.